UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

ACTIVIDAD 4. Algoritmos de búsqueda ciega: BFS y DFS

Asignatura: Fundamentos de Inteligencia Artificial

Docente: Mtro. Javier García Lara

Carrera: Ingeniería en Tecnologías de la Información e Innovación Digital

Grupo: 171 **Cuatrimestre:** 7

Alumno: Suarez Melonie Magdalena

1. Introducción

Los algoritmos de búsqueda ciega permiten encontrar soluciones sin utilizar información adicional o heurística sobre qué tan cerca se encuentra la meta. Para explorar los estados, siguen un orden determinado y registran los nodos que ya han visitado.

En esta actividad se implementan BFS (Breadth-First Search) y DFS (Depth-First Search) para resolver un laberinto. Se comparan sus caminos, la cantidad de nodos expandidos y el tiempo de ejecución para comprender sus diferencias.

2. Planteamiento del problema

Se tiene un laberinto representado mediante una cuadrícula. El agente debe encontrar una ruta desde el punto inicial S hasta la meta M. Puede moverse hacia arriba, abajo, izquierda o derecha, pero no puede atravesar paredes.

El objetivo es implementar BFS y DFS, comprobar si ambos encuentran una solución y comparar su desempeño.

3. Representación del espacio de estados

Cada celda transitable se representa con una coordenada formada por la fila y la columna.

Cada celda transitable es un estado y cada movimiento posible hacia una celda vecina representa una conexión del grafo.

4. Descripción de los algoritmos

4.1 BFS búsqueda en anchura

BFS utiliza una cola de tipo FIFO, es decir, el primer nodo que entra es el primero en salir. Explora primero los nodos más cercanos al inicio y después continúa con los siguientes niveles.

Cuando todos los movimientos tienen el mismo costo, BFS encuentra una ruta con el menor número de movimientos.

4.2 DFS búsqueda en profundidad

DFS utiliza una pila de tipo LIFO, por lo que el último nodo agregado es el primero que se explora. El algoritmo continúa por una ruta hasta que no puede avanzar y después retrocede para explorar otras opciones.

DFS puede encontrar una solución sin explorar todos los niveles, pero no garantiza encontrar la ruta más corta.

5. Código Python

```python
from collections import deque
import time

# S = inicio, M = meta, # = pared, . = camino
laberinto = [
    ["S", ".", ".", "#", ".", "."],
    ["#", "#", ".", "#", ".", "#"],
    [".", ".", ".", ".", ".", "."],
    [".", "#", "#", "#", "#", "."],
    [".", ".", ".", ".", ".", "M"]
]

inicio = (0, 0)
meta = (4, 5)

# Abajo, derecha, arriba e izquierda
MOVIMIENTOS = [(1, 0), (0, 1), (-1, 0), (0, -1)]


def buscar(metodo):
    if metodo == "BFS":
        pendientes = deque([inicio])
    else:
        pendientes = [inicio]

    padres = {inicio: None}
    expandidos = 0

    while pendientes:
        if metodo == "BFS":
            actual = pendientes.popleft()
        else:
            actual = pendientes.pop()

        expandidos += 1

        if actual == meta:
            ruta = []
            nodo = actual

            while nodo is not None:
                ruta.append(nodo)
                nodo = padres[nodo]

            ruta.reverse()
            return ruta, expandidos

        if metodo == "BFS":
            movimientos = MOVIMIENTOS
        else:
            movimientos = list(reversed(MOVIMIENTOS))

        for df, dc in movimientos:
            nf = actual[0] + df
            nc = actual[1] + dc
            vecino = (nf, nc)

            dentro = (
                0 <= nf < len(laberinto)
                and 0 <= nc < len(laberinto[0])
            )

            if (
                dentro
                and laberinto[nf][nc] != "#"
                and vecino not in padres
            ):
                padres[vecino] = actual
                pendientes.append(vecino)

    return None, expandidos


def mostrar_ruta(ruta):
    dibujo = [fila[:] for fila in laberinto]

    for fila, columna in ruta[1:-1]:
        dibujo[fila][columna] = "*"

    for fila in dibujo:
        print(" ".join(fila))


if __name__ == "__main__":
    for metodo in ("BFS", "DFS"):
        inicio_tiempo = time.perf_counter()

        ruta, expandidos = buscar(metodo)

        tiempo = time.perf_counter() - inicio_tiempo

        print(f"\nRESULTADOS DE {metodo}")

        if ruta:
            print("Ruta encontrada:", ruta)
            print("Movimientos:", len(ruta) - 1)
            print("Nodos expandidos:", expandidos)
            print(f"Tiempo: {tiempo:.6f} segundos")
            print("Laberinto con la ruta marcada:")
            mostrar_ruta(ruta)
        else:
            print("No se encontró una solución.")
```

6. Procedimiento de implementación

1. Se representa el laberinto como una lista de listas.
2. Se establecen las coordenadas del inicio y la meta.
3. Se definen los movimientos permitidos.
4. Se crea una función que implementa BFS mediante una cola y DFS mediante una pila.
5. Se utiliza un diccionario para registrar los padres de cada estado y reconstruir la ruta.
6. Se cuentan los nodos expandidos y se mide el tiempo de ejecución.
7. Se imprimen las rutas y el laberinto con el camino encontrado marcado con *.

7. Resultados y comparación

El programa presenta los resultados reales de ambos algoritmos al ejecutarse. La comparación debe completarse con los datos obtenidos en tu computadora.

Criterio	BFS	DFS
Estructura utlizada	Cola FIFO	Pila LIFO
Forma de explorar	Por niveles	Por profundidad
Encuentra una ruta	Sí, si existe	Sí, si existe y la búsqueda alcanza la meta
Garantiza el camino más corto	Sí, con costos iguales	No
Tiempo	Se mide durante la ejecución	Se mide durante la ejecución
Nodos expandidos	Se cuentan durante la busqueda	Se cuentan durante la búsqueda

Los tiempos de ejecución pueden ser muy pequeños y variar según el equipo. Por ello, conviene utilizar los valores que imprima el programa y no inventar resultados.

8. Análisis de complejidad

Si el laberinto se representa como un grafo con V vértices y E aristas, tanto BFS como DFS tienen una complejidad temporal de:

$$O(V + E)$$

Esto significa que, en el peor caso, ambos pueden recorrer todos los estados alcanzables y sus conexiones.

BFS puede necesitar más memoria porque mantiene una cola con los nodos pendientes de explorar. DFS utiliza una pila y puede profundizar en una ruta antes de regresar a otras opciones. El desempeño real depende de la estructura del laberinto y del orden de exploración.

9. Conclusión

En esta actividad se implementaron dos algoritmos de búsqueda ciega para resolver un laberinto. Aunque ambos pueden encontrar una solución, utilizan estrategias diferentes para explorar los estados.

BFS es conveniente cuando se necesita encontrar la ruta más corta y todos los movimientos tienen el mismo costo. DFS puede resultar útil para explorar una ruta en profundidad, aunque no garantiza que la solución encontrada sea la más corta.

La comparación de los nodos expandidos y el tiempo de ejecución permite comprender mejor cómo influye la estrategia de búsqueda en la resolución de un problema.

10. Declaración de uso de inteligencia artificial

Se utilizó ChatGPT como apoyo para organizar la explicación del tema y elaborar una estructura inicial del código

11. Referencias

Georgiou, I. (2026). SUNSPOT PREDICTION WITH a NEURAL NETWORK: AN APPLICATION OF ADVANCED ARTIFICIAL INTELLIGENCE IN SECONDARY EDUCATION. *International Journal Of Global Education (IJGE)*. http://www.ijge.net/index.php/php/article/view/84

Tan, J., Feitosa, D., Avgeriou, P., & Lungu, M. (2020). Evolution of technical debt remediation in Python: A case study on the Apache Software Ecosystem. *Journal Of Software Evolution And Process*, *33*(4). https://doi.org/10.1002/smr.2319